Springleik: A Case Study in Small Languages

Mark S. Williamsen

July 17, 2026

Abstract

Small languages arise in computing for a variety of reasons. A simple command interpreter can be useful in bringing up new hardware, particularly on small targets running embedded firmware. This leads naturally to a scriptable interface. Which can then evolve further into a primitive domain-specific language (DSL) that supports use cases in development, test, and even production. This work describes one path through the steps that lead from command interpreter to small language. Such languages can be very efficient with the resources they use including memory, processor bandwidth, and system services. Memory allocations can be static or dynamic, according to the details of the application. Additional features including self-documenting code and generation of baseline test outputs are easily implemented with minimal effort.

1 Introduction

2 Model of software development

Many models are available for software development for this discussion. Here we present a simple model based on closed loop control theory, which includes concepts of time dependence and iteration. Figure 1 is a simulation diagram representing a software development process. The input on the left is requirements $R(s)$, and the output on the right is executable code and other deliverable assets $D(s)$. In between we have a means for creating code *Implement* positioned as the plant in control theory, and a means for testing code *Test* positioned as the sensor. The summing junction on the left compares test results $V(s)$ with requirements $R(s)$ to produce an error term $E(s)$, which then drives ongoing development. The directed edges connecting these parts together represent N-dimensional vectors where N is the number of requirements. The value of each dimension is the extent to which a given requirement has been met on some arbitrary scale.

A typical software project will begin with all vectors pointing to the origin: no requirements, no implementation, and no tests. As development proceeds we add requirements which drive the implementation, which is then tested and compared with requirements. Each directed edge conveys important information, but the most interesting vector is the error term $E(s)$. This will wander in an N-dimensional problem space, with N steadily increasing as development proceeds. Ideally this vector should always trend towards zero, with distance from the origin giving us an indication of how far we are from done. Time

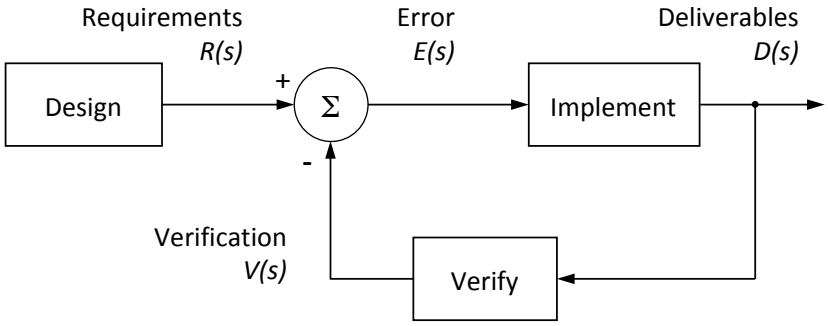

Figure 1: Software development process rendered as a closed-loop control simulation diagram. Negative feedback from the *Test* block causes the *Deliverables* to evolve, tracking changes in *Requirements*. When all requirements have been met, the error term $E(s)$ goes to zero and the control loop is in steady state.

dependence is not shown explicitly but one can imagine one or more integrators in the loop, with time constants we may or may not control, modifying the time domain response of the development process.

Treating the development process as continuous we can write an expression for deliverables $D(s)$ in terms of implementation $I(s)$, test $T(s)$, and requirements $R(s)$, where s is the Laplace variable.

$$D(s) = R(s)\frac{I(s)}{1 + I(s)T(s)} \tag{1}$$

In this model we see that there are three things which matter: requirements, implementation, and tests. These pillars of software development should receive equal priority during development, so that all three can be considered as correct and complete when development ends and maintenance phase begins. As a practical matter however many projects end with one or more of the pillars incomplete. I claim without proof that given any one of these, the other two can be derived through various means. For the purposes of this paper we use the simulation diagram in figure 1 to define deriving implementation and tests from requirements as "forward engineering." Starting with an implementation or a test suite is then defined as "reverse engineering."

3 Object model for small targets

Developers of embedded software for small targets may be tempted to organize their object model around things the system has, such as inputs, outputs, sensors, and actuators. I would submit however that a coherent design is more likely to result from an object model organized around things the embedded system does, such as "home an axis", "measure a sample", "acquire a trace", or "energize a solenoid." The real reason we think in terms of objects is so that we can throw them into a container and iterate over them. While there may be a use case for iterating over peripherals, the important use case here is

iterating over a collection of nodes which represent actions a user might want to invoke. These are packaged as command nodes along with the arguments needed to carry out each action. Thus we replace a noun-oriented design with a verb-oriented design so that command nodes can be collected into sequences which can then be interpreted on live hardware, simulated hardware, or a hybrid of both. Let's look at an example of how such a design might arise.

Consider a small target with embedded processor and several axes of motion control. On first bring-up we want to start characterizing the motion controller. We write a small monitor program to run on the target with a command prompt exposed on a serial port, or a socket port if so equipped. Then we write commands to initialize the motion controller, home an axis, move axis to a new position, and wait for position reached. Each command can be followed with one or more arguments giving details such as velocity, acceleration, current, position, etc. Running a terminal program on a host computer allows us to manually exercise various behaviors of the motion controller. We can stream a text file containing motion commands from the host to the target's monitor program to execute a motion sequence of arbitrary complexity. Move commands however take finite time, and we wind up overrunning the monitor program with new commands before old ones are complete. So we write a program to run on the host and carry out a series of motions on the target using monitor program commands, but now waiting for a prompt from the target before sending a new command from the host.

4 Minimum footprint

Many readers will have gone through these steps in bringing up a new target. We don't yet have a small language, but many of the pieces needed are already in place. Now we implement our object model, starting on the host side, declaring a class object for each command and an instance attribute for each argument the command needs. One can also add an instance attribute for the command's return value, if that value is meaningful. Now we have a small language we can use to carry out the following steps:

- Instantiate one or more class objects representing command nodes to be executed.
- Populate the nodes with arguments appropriate for the desired commands.
- Compose the nodes into a program by appending them to a collection.
- Iterate over the collection, interpreting each node as a command with arguments.
- Capture results returned for each command node, either as attributes of the command nodes or in a separate trace or listing.

Let's look at what we have. The class objects representing command nodes can live almost anywhere: on the host, on the target, in the cloud, or anywhere else that can interpret them into commands with arguments. For now we think of the arguments as being hard-coded, but they could just as well be derived or symbolic. The type of collection used to iterate over the command nodes doesn't

```
[
  {"cmd": "initControl"},
  {"cmd": "setRunCurrent",  "axis": 1, "current_mA": 500},
  {"cmd": "setIdleCurrent", "axis": 1, "current_mA": 500},
  {"cmd": "setVelocity",    "axis": 1, "steps_sec": 100},
  {"cmd": "setAccel",       "axis": 1, "steps_sec2": 100},
  {"cmd": "homeAxis",       "axis": 1},
  {"cmd": "setPosition",    "axis": 1, "steps": 200},
  {"cmd": "waitPosition",   "axis": 1},
  {"cmd": "setPosition",    "axis": 1, "steps": 0},
  {"cmd": "waitPosition",   "axis": 1}
]
```

Figure 2: JSON representation of a simple command list being sent to an embedded motion controller. Arguments here are named, but could also be positional.

matter much except that iteration should follow a predictable, repeatable path. The commands themselves can be sent to one or more target devices, or to any other device or simulation with an interface to receive them. Command replies such as motion results or error messages in our motion control example can go back to the host, to some other monitor, or even be ignored. This then is the bare minimum of what's needed to claim we have a small domain-specific language. This can be prototyped and running in a matter of a few days, completely avoiding the trap that "nothing works until everything works."

5 Serialization

Now let's look at what's missing. To obtain a text representation of the program we'll need a means to serialize the collection of command nodes. We have many well-established choices such as JSON, XML, YAML, etc. We also have the freedom to design our own representation if warranted, or dispense with source code altogether. In this paper we will assume without loss of generality that JSON has been chosen. Experience shows that it is trivial to give command node classes a serialize method which is specialized for specific commands and arguments. Instead of iterating over the collection while executing each node, we iterate over the collection while serializing each node to obtain a program listing. If we have multiple interpreters (host, target, cloud, etc.) all should produce identical program listings for a given collection of command nodes. Returning to the example of an embedded motion controller we can look at what the serialized JSON representation might look like in figure 2. It should be mentioned here that while off-target interpreters can use the monitor command prompt interface, on-target interpreters don't have to since they can call directly into functions on the target.

Real programming however implies branching and looping, which is easily added by defining control nodes for conditional and repeated execution where the arguments represent loop count, limits for comparison, etc. At this point we've only described programs which are a simple list or sequence of nodes. Now we need to expand our definition of command nodes to include a node list attribute, allowing us to represent hierarchical tree structures. Not every node

```
{"cmd": "initControl", "list": [
  {"cmd": "setCurrent",  "axis": 1, "current_mA": 500},
  {"cmd": "homeAxis",    "axis": 1},
  {"cmd": "setVelocity", "axis": 1, "steps_sec": 100},
  {"cmd": "setAccel",    "axis": 1, "steps_sec2": 100},
  {"cmd": "setCurrent",  "axis": 2, "current_mA": 250},
  {"cmd": "homeAxis",    "axis": 2},
  {"cmd": "setVelocity", "axis": 2, "steps_sec": 50},
  {"cmd": "setAccel",    "axis": 2, "steps_sec2": 50},
  {"cmd": "iterateRegister", "reg": "$1",
    "start": 0, "stop": 200, "step": 20, "list": [
    {"cmd": "setPosition", "axis": 1, "steps": "$1"},
    {"cmd": "waitPosition", "axis": 1},
    {"cmd": "iterateRegister", "reg": "$2",
      "start": 0, "stop": 100, "step": 10, "list": [
      {"cmd": "setPosition", "axis": 2, "steps": "$2"},
      {"cmd": "waitPosition", "axis": 2}
    ]}
  ]},
  {"cmd":"testInput", "digIn": 1, "trueIf": 1, "list":[
    {"cmd": "setCurrent",  "axis": 1, "current_mA": 50},
    {"cmd": "setCurrent",  "axis": 2, "current_mA": 25}
  ]}
]}
```

Figure 3: JSON representation of a command tree with conditional and repeating nodes. The root node is both a dictionary and a command node. But we could just as well have subclassed *branch* to have a *root* node class. Or used a base class node, with its default behaviors. These are design decisions, along with many other details. $1 and $2 are symbols for local registers visible to subordinates of the "iterateRegister" nodes.

needs to have a list. So we divide node classes into composite or *branch* nodes which have a list, and terminal or *leaf* nodes which don't. Clearly conditional and repeating nodes should have lists, to which we append nodes that should only be executed if the condition evaluates as true, or which will be repeated in iterations controlled by a repeating node. Our class hierarchy can begin with the base class *node* which has two subclasses, *branch* and *leaf*. Command and control nodes can then be derived from *branch* and *leaf*. All node classes should have methods to execute and to serialize them while traversing a command tree by recursive descent. The execute and serialize methods of composite nodes must also iterate over the node list, calling the execute or serialize method of each subordinate node in turn. The execute method of a composite node can include entry and exit code, to run before and after iterating over the node list.

In figure 3 we look at the JSON representation of a command tree which includes conditional and repeating nodes. Iteration is done by the new "iterateRegister" control node which changes the value of a numbered register which is then visible to nodes beneath the control node. Nested loops are supported here by placing the iteration of axis 2 inside the iteration of axis 1. At the end of the sequence a "testInput" control node tests a digital input to check whether

to return to holding current.

The ability to implement tree structures composed of command and control nodes, using all of the infrastructure we've developed so far, brings into focus a scalable, portable, and verifiable framework for representing interpreted programs of arbitrary size, complexity, and capability. Now we have an abstract syntax tree (AST) interpreter optimized for the application at hand. The composite tree structure maps directly to structured text representations such as JSON, XML, or YAML. Human-readable program listings are easily obtained by serializing nodes while traversing the tree with recursive descent. We encounter each node exactly once when we ask the root node to serialize the command tree. In this picture we can now say clearly that command nodes are dictionaries composed of key-value pairs representing all of the command arguments, along with an ordered list container holding pointers or references to subordinate nodes.

6 Deserialization

There are several features we take for granted in established programming languages which might not be needed in a small bespoke language. One is deserialization to parse a structured text representation into command nodes in memory. Another is the ability to parse and execute general math expressions. Experience shows that each of these features imposes a significant burden of time and expense to implement, document, and verify. Readers should consider how much value would be added if they were to go down this path. Note that these features will likely be implemented in two steps, first parsing the text representation into a generic syntax tree, and then rendering the syntax tree as domain-specific command nodes to be interpreted at run-time.

Command tree programs which rarely change can be composed of statically allocated nodes. Programs that could change at run-time however will be composed of nodes taken from a node pool, or using dynamically allocated memory. In garbage-collected environments dynamic command nodes will evaporate after they are no longer referenced. Releasing tree nodes without garbage collection is easy in that we can ask the root node to delete itself and all subordinates by recursive descent. For this to work each node must appear in the tree exactly once.

7 Execution model

Should I bring in threads, tasks, and processes? Or is that out of scope?

8 Analysis

Say something about decorating the tree with measurement and analysis nodes. Which should lead naturally to a discussion of separation of execution and analysis.

We'll discuss deserialization more when we consider adding an analysis phase to our command tree structure.

9 Vocabulary

I used to value lexical, grammatical, semantical, and syntactical thinking.

Say something about granularity of command node objects. My guidance would be to package command nodes in the same sense as you would package monitor prompt commands. Problem solved!

10 Verification

Test and verification will lead naturally to my concept of an asymmetric tree comparison.

11 Communication

Say something about rendering the tree in various human and/or machine readable forms. Ideally one-to-one mapping in all directions. With the possibility to run round-trip testing.

Say something about machine-readable versus human-readable source code, bringing AI development and maintenance into the conversation.

Bring into the picture how AI can develop, verify, and maintain AST command trees. Make the point that flow charts don't help to communicate with machines, but pseudocode does.

12 Toy model

When to introduce toy motor? Should I use it for everything? I'd like to modify the toy motor demo to include a monitor prompt command interpreter. Should I use a console, a socket port, named pipe, anonymous pipe, or what? The obvious solution is to use my multi-client multi-server code already posted on GitHub. Note well that I can mix and match environments, running clients in Python, Java, C, etc. and also running servers in Python, Java, C, etc.

For the sake of discussion I've used the example of small embedded targets used for measurement/control/communication. But these techniques have much broader application, and are in fact universal.